

Báo cáo bài tập EX8: model_selection

5. Bài tập ¶

- Sử dụng dữ liệu đánh giá tín dụng cá nhân
- Sử dụng độ đo đánh giá negative cost
- Lựa chọn tham số cho các mô hình SVM, Random Forest và KNN
- So sánh các mô hình với siêu tham số tốt nhất

```
: url = "https://archive.ics.uci.edu/ml/machine-learning-databases/statlog/german/german.data-numeric"
data = np.genfromtxt(url)
X_data = data[:, :24]
Y_data = data[:, -1]
print(X_data.shape)
print(Y_data.shape)
```

```
(1000, 24)
(1000,)
```

```
In [19]: # Hàm tính neg_cost, dùng để truyền vào scoring của learning_curve
def neg_cost(estimator, X, y):
    y_true = y
    y_pred = estimator.predict(X)
    true_pos = ((y_true==y_pred)&(y_true==1.0))*0.0
    true_ne = ((y_true==y_pred)&(y_true==2.0))*0.0
    false_ne = ((y_true!=y_pred)&(y_true==1.0))*1.0
    false_pos = ((y_true!=y_pred)&(y_true==2.0))*5.0
    return -sum(true_pos + true_ne + false_pos + false_ne)/len(y_true)
```

```
In [20]: # train_test_split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X_data, Y_data, test_size=0.2, random_state=42)
```

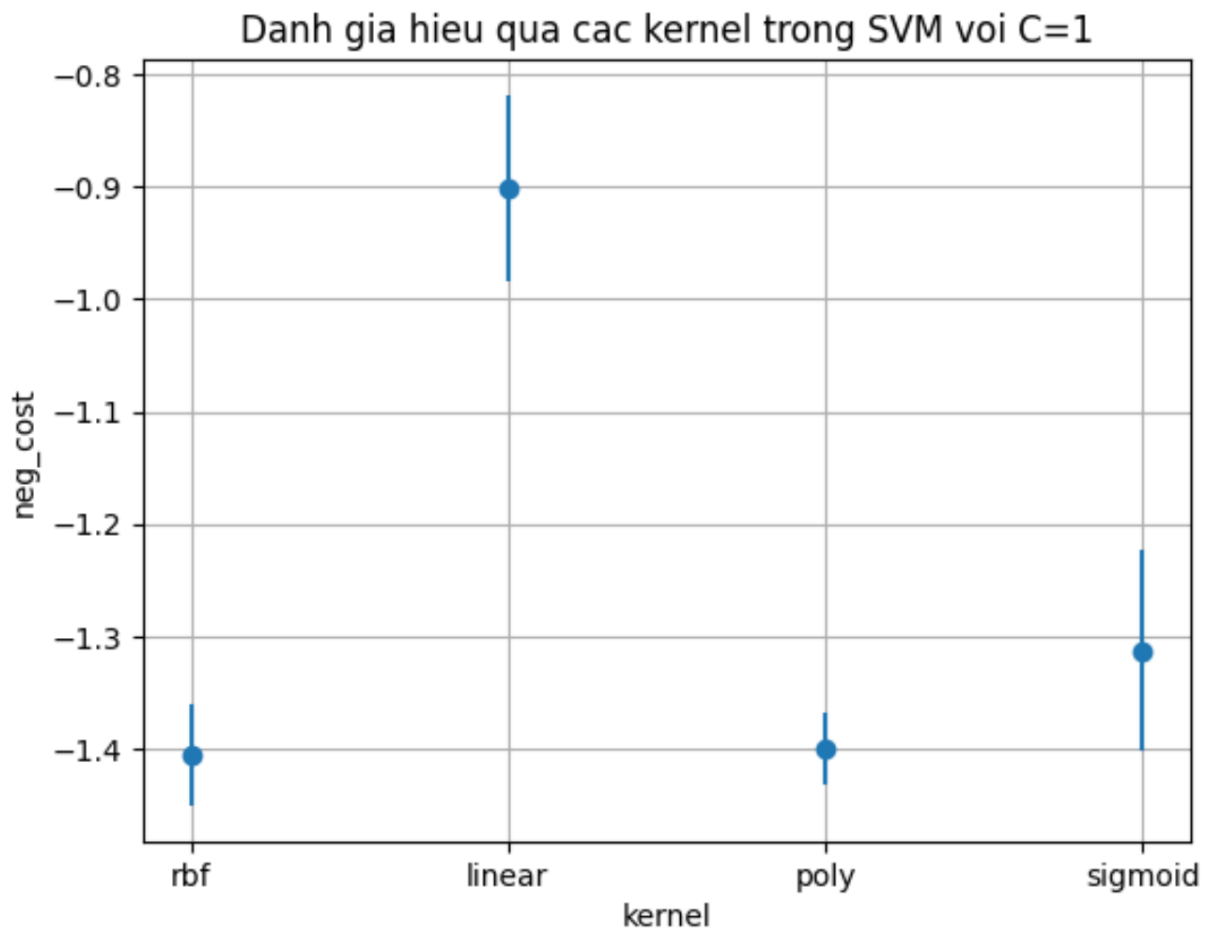
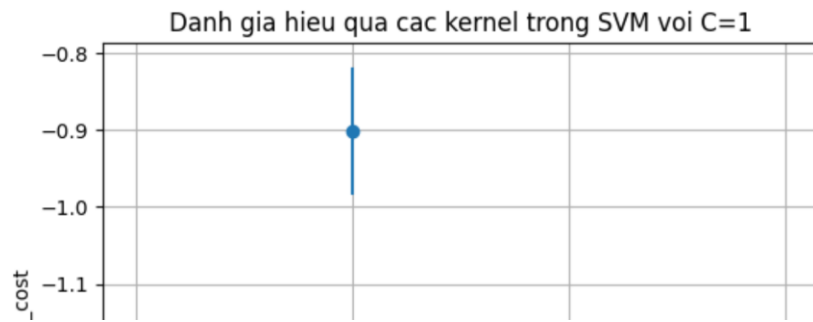
```
In [21]: # cross_validation và vẽ hình
def cross_validation(estimator):
    _, _, test_scores = learning_curve(estimator, X_train, y_train, n_jobs=-1, train_sizes=[1.0, ], sc
    test_score=test_scores[0]
    mean, std=test_score.mean(), test_score.std()
    return mean, std

def plot(title, xlabel, X, Y, error, ylabel='neg_cost'):
    plt.title(title)
    plt.xlabel(xlabel)
    plt.ylabel(ylabel)
    plt.errorbar(X, Y, error, linestyle='None', marker='o')
    plt.grid()
```

```
In [22]: # danh gia hieu qua cac kernel trong SVM
title="Danh gia hieu qua cac kernel trong SVM voi C=1"
x=[]
y=[]
error=[]
for kernel in tqdm(['rbf', 'linear', 'poly', 'sigmoid']):
    model=svm.SVC(kernel=kernel, C=1)
    mean, std=cross_validation(model)
    x.append(kernel)
    y.append(mean)
    error.append(std)

plot(title, "kernel", x, y, error)
```

100% | 4/4 [00:00
<00:00, 4.89it/s]

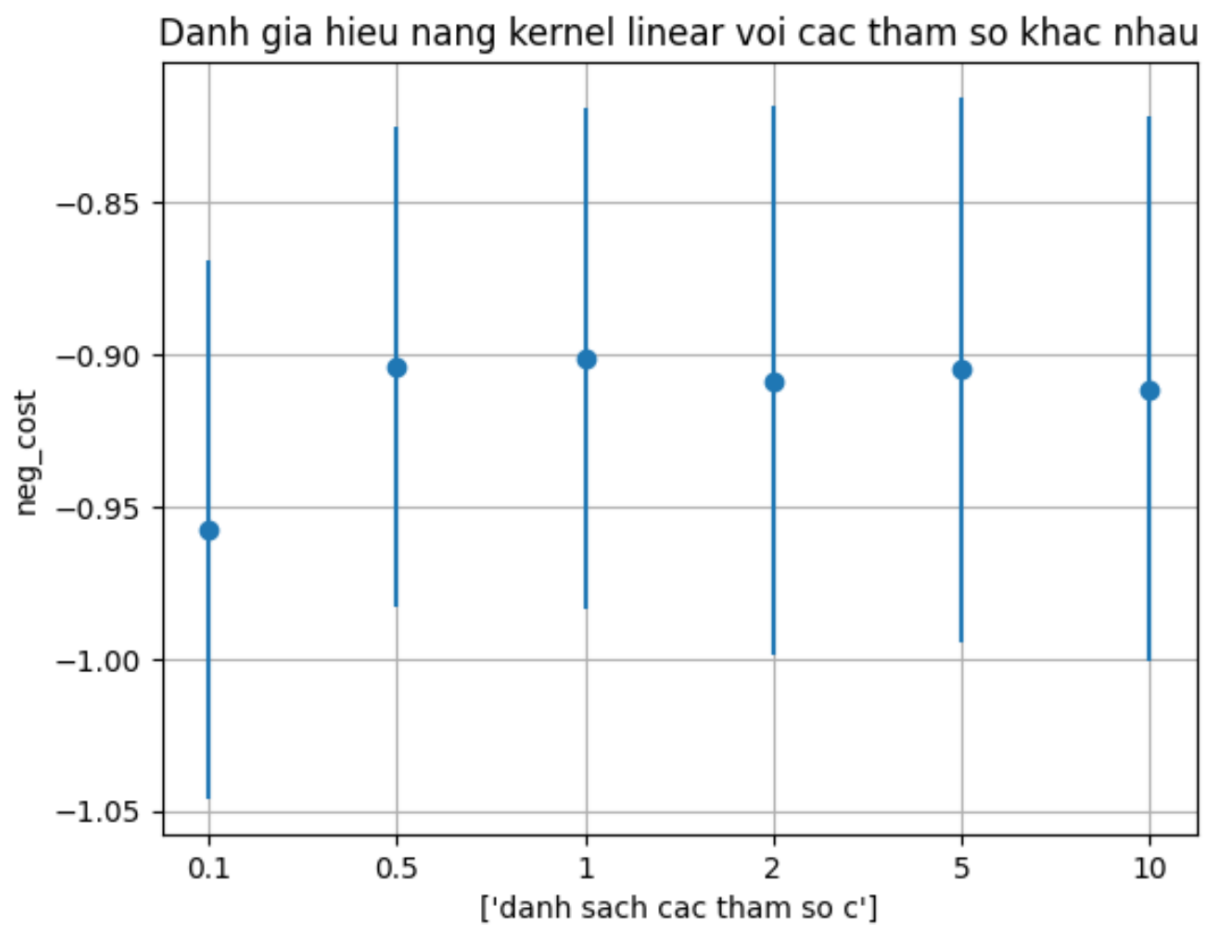


```
title="Danh gia hieu nang kernel linear voi cac tham so khac nhau"
x=[]
y=[]
xtitle=['danh sach cac tham so c']
error=[]
```

```
for c in tqdm([0.1, 0.5, 1, 2, 5, 10]):
    model=svm.SVC(kernel='linear', C=c)
    mean, std=cross_validation(model)
    x.append(str(c))
    y.append(mean)
    error.append(std)
```

```
plot(title, xtitle, x, y, error)
```

100% | 6/6 [00:24
<00:00, 4.01s/it]

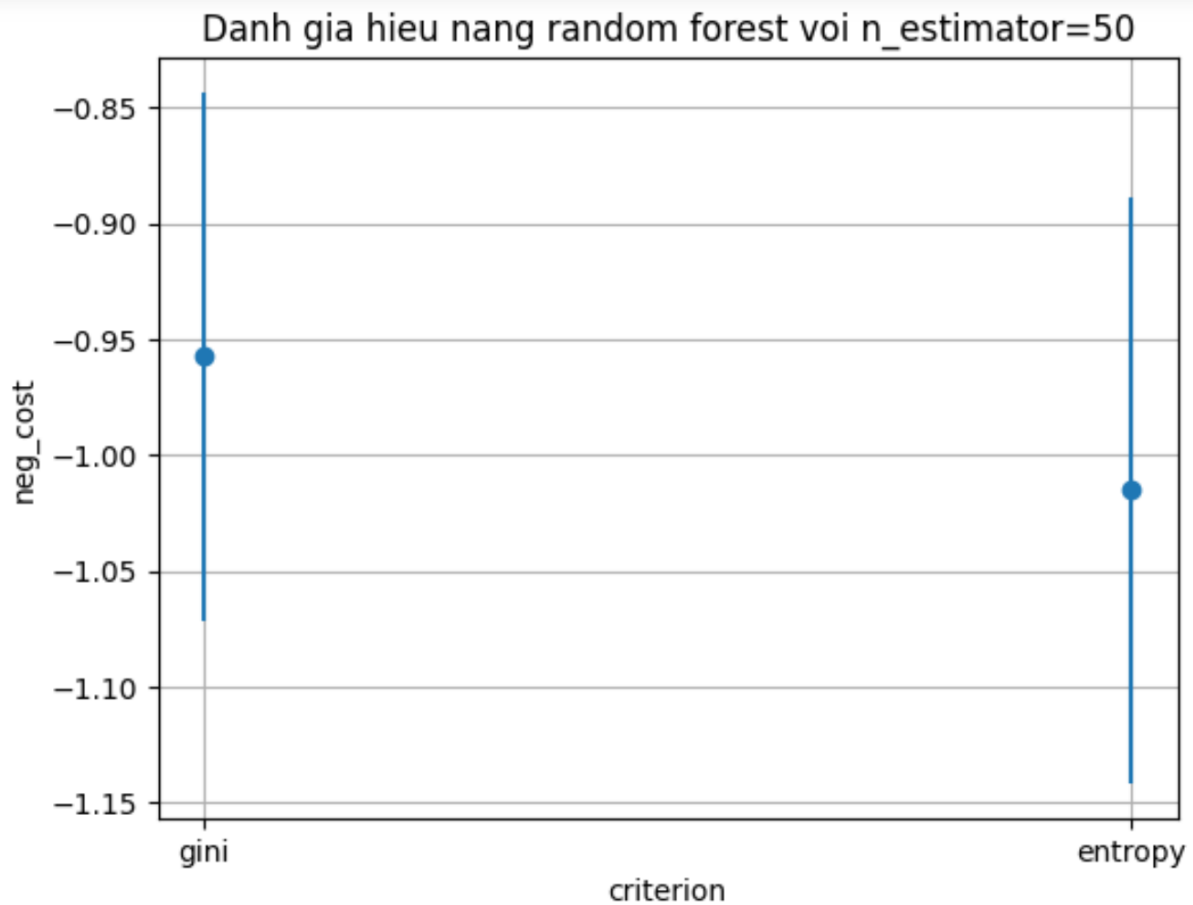


```

title="Danh gia hieu nang random forest voi n_estimator=50"
xlabel="criterion"
x=[]
y=[]
error=[]
for criterion in tqdm(['gini', 'entropy']):
    model=RandomForestClassifier(criterion=criterion, n_estimators=50)
    mean, std=cross_validation(model)
    x.append(str(criterion))
    y.append(mean)
    error.append(std)
plot(title, xlabel, x, y, error)

```

100% | 2/2 [00:00
<00:00, 6.45it/s]



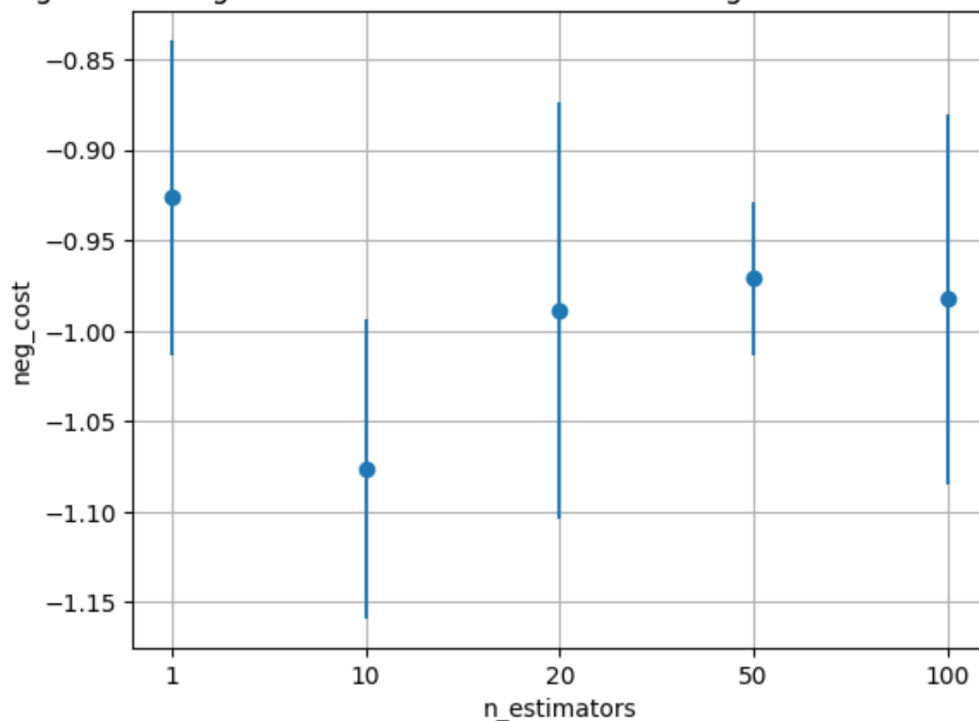
```

title="Danh gia hieu nang cua random forest voi criterion la gini theo tham so n_estimators"
xlabel="n_estimators"
x=[]
y=[]
error=[]
for n_estimators in tqdm([1, 10, 20, 50, 100]):
    model=RandomForestClassifier(criterion='gini', n_estimators=n_estimators)
    mean, std=cross_validation(model)
    x.append(str(n_estimators))
    y.append(mean)
    error.append(std)
plot(title, xlabel, x, y, error)

```

100% | 5/5 [00:00
<00:00, 10.55it/s]

Danh gia hieu nang cua random forest voi criterion la gini theo tham so n_estimators



```

title = "danh gia hieu nang voi thuat toan knn thay doi K"
xlabel = "K"
X = []
Y = []
error = []

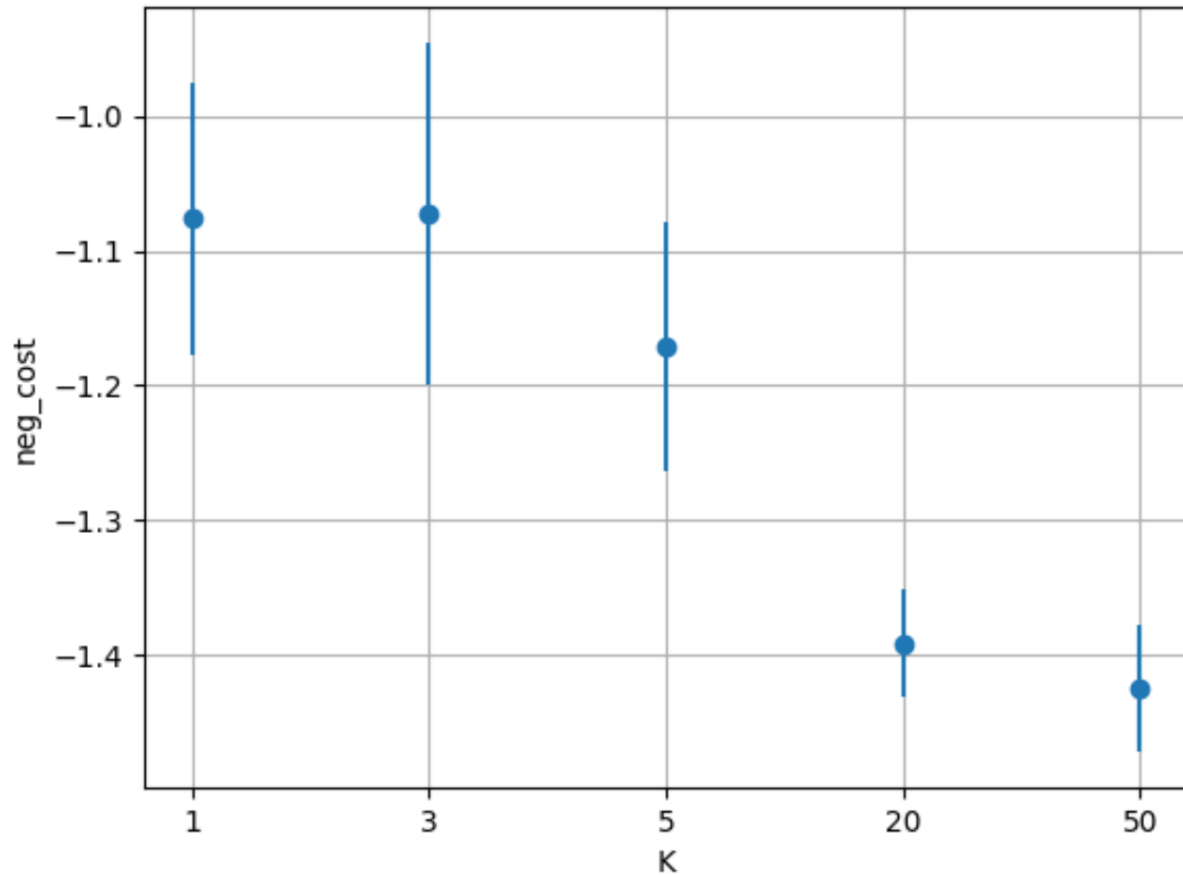
for k in tqdm([1, 3, 5, 20, 50]):
    text_clf = KNeighborsClassifier(n_neighbors=k)
    mean, std = cross_validation(text_clf)
    X.append(str(k))
    Y.append(mean)
    error.append(std)

plot(title, xlabel, X, Y, error)

```

100% | 5/5 [00:00
<00:00, 13.36it/s]

đánh giá hiệu năng với thuật toán knn thay đổi K



```
## so sánh lại các thuật toán với tham số tốt nhất
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X_data, Y_data, test_size=0.2, random_state=42)
y_test.shape
```

(200,)

```
rf=RandomForestClassifier(criterion='gini', n_estimators=30)
knn=KNeighborsClassifier(n_neighbors=3)
svm_=svm.SVC(kernel='linear', C=5)
```

```
rf.fit(X_train, y_train)
knn.fit(X_train, y_train)
svm_.fit(X_train, y_train)
```

▼ SVC ⓘ ?

Parameters

```
rf.fit(X_train, y_train)
knn.fit(X_train, y_train)
svm_.fit(X_train, y_train)
```

▼ SVC ⓘ ?

Parameters

```
print(accuracy_score(y_test, rf.predict(X_test)))
print(accuracy_score(y_test, knn.predict(X_test)))
print(accuracy_score(y_test, svm_.predict(X_test)))
```

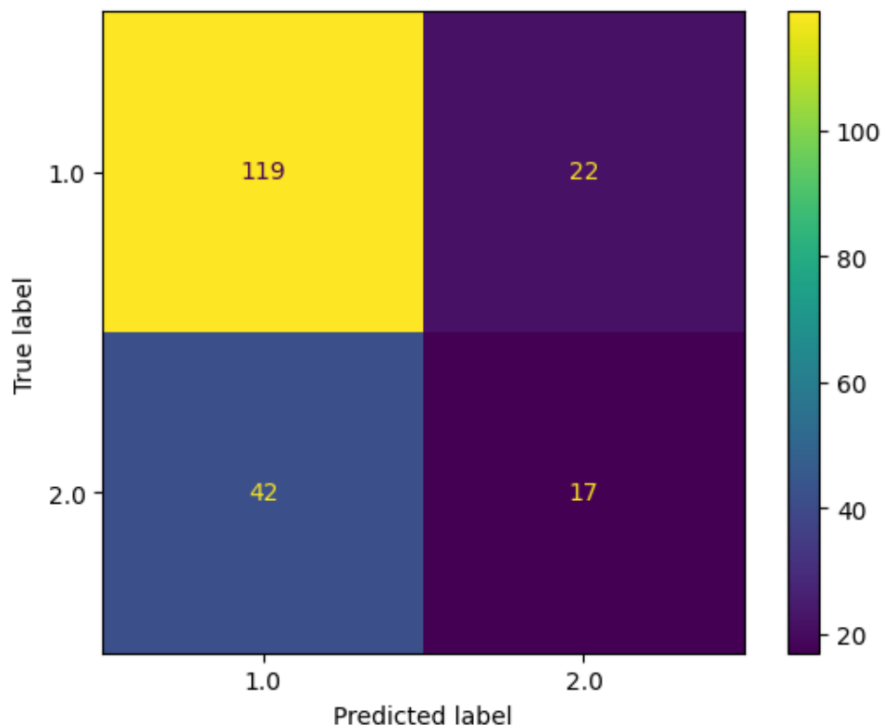
0.78

0.68

0.76

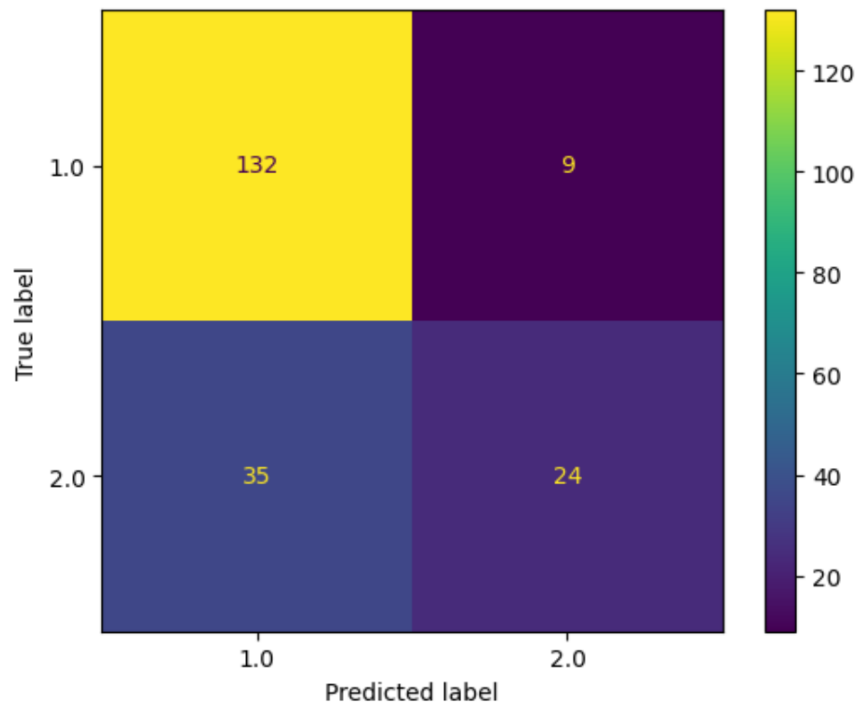
```
ConfusionMatrixDisplay.from_estimator(knn, X_test, y_test)
```

<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x25371d796e0>



```
ConfusionMatrixDisplay.from_estimator(rf, X_test, y_test)
```

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x25371f2c710>
```



```
ConfusionMatrixDisplay.from_estimator(svm_, X_test, y_test)
```

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x25371d79350>
```

